Alexandria Engineering Journal 140 (2026) 33–46

Contents lists available at ScienceDirect

Alexandria Engineering Journal

journal homepage: www.elsevier.com/locate/aej

Original article

# Enhancing UAV path planning with diffusion models and group relative policy optimization

Yuyue Liu [a], Shuzhi Liu [a,b,*], Zhongyi Wang [a], Chengshi Yuan [a], Xinyu Guo [a], Yujian Liu [a], Jixiang Wang [a]

[a] School of Physics and Electronic Engineering, Qilu Normal University, Jinan, Shandong 250200, China
[b] Shandong Key Laboratory of Space Environment and Exploration Technology, Jinan, Shandong 250200, China

ARTICLE INFO

*Keywords:*
Unmanned aerial vehicles
Diffusion strategy
Deep reinforcement learning
Group relative policy optimization
Control barrier function

ABSTRACT

This paper addresses the uncertainties arising from dynamic environments in autonomous path planning tasks for unmanned aerial vehicles (UAVs). Existing Deep Reinforcement Learning (DRL) methods struggle to represent multiple feasible trajectories and often neglect failure experiences. To overcome these limitations, we propose a novel framework termed GRPO-MDP (Group Relative Policy Optimization with a Multimodal Diffusion Strategy). The framework leverages a Denoising Diffusion Probabilistic Model (DDPM) to generate diverse action samples during training. By integrating DRL with multimodal diffusion strategies, GRPO-MDP enables UAVs to explore multiple feasible trajectories from the same state, enhancing exploration flexibility. The proposed Group Relative Policy Optimization (GRPO) method constructs advantage functions through relative comparisons within groups of trajectories, thereby eliminating the need for a value network and improving training stability. In addition, a hindsight trajectory relabeling mechanism is introduced to convert failure experiences into informative learning signals by incorporating virtual target and safety boundary modes. To guarantee real-time safety, a Control Barrier Function (CBF) is employed as a safety filter, ensuring that hard safety constraints are strictly enforced during path planning. Experimental results demonstrate that the proposed GRPO-MDP framework outperforms existing methods in terms of success rate, path efficiency, and safety in dynamic environments.

## 1. Introduction

Unmanned Aerial Vehicles (UAVs) are remotely controlled aircraft that, compared to traditional manned aircraft, are smaller, more cost-effective, and capable of performing a wide range of tasks, including agriculture, environmental monitoring, and military reconnaissance [1]. Autonomous UAV path planning refers to the UAV's ability to autonomously generate flight trajectories, avoid obstacles, select optimal routes, and dynamically adjust its flight strategy based on real-time environmental perception and predefined objectives, all without human intervention. However, in real-world environments, UAVs face significant challenges due to their limited perception ranges and the presence of numerous obstacles, making trajectory planning difficult [2].

Autonomous UAV path planning methods can be broadly classified into four categories: sampling-based, optimization-based, imitation learning-based, and Deep Reinforcement Learning (DRL)-based methods. Sampling-based planning methods, such as Rapidly-exploring Random Trees (RRT) [3] and Probabilistic Roadmaps (PRM), construct path graphs by sampling in the state space [4]. Although effective in certain applications, these methods suffer from high computational complexity, particularly in high-dimensional spaces, making them unsuitable for real-time applications. Additionally, the paths generated are often not smooth, necessitating extensive post-processing for practical implementation. Optimization-based methods approach the problem by solving for the optimal control sequence within a finite time horizon using rolling optimization techniques [5,6]. While these methods can yield precise solutions, they are computationally expensive and heavily reliant on accurate system dynamics models, limiting their applicability in rapidly changing or uncertain environments. Imitation learning-based methods learn policies by mimicking expert demonstrations [7,8]. However, these methods depend on the quality of the expert data, exhibiting limited generalization when faced with scenarios outside the training distribution, and often fail to outperform expert performance.

* Corresponding author at: School of Physics and Electronic Engineering, Qilu Normal University, Jinan, Shandong 250200, China.
*E-mail address:* shuzhiliu@qlnu.edu.cn (S. Liu).

https://doi.org/10.1016/j.aej.2026.02.024
Received 7 November 2025; Received in revised form 16 January 2026; Accepted 22 February 2026
Available online 25 February 2026
1110-0168/© 2026 The Author(s). Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

In recent years, DRL has emerged as a promising solution for UAV autonomous navigation, offering a more flexible approach for decision-making tasks. DRL-based methods can be categorized into value function-based, policy gradient-based, and model-based reinforcement learning approaches. Value function-based methods guide decision-making by learning a state-action value function [9,10]. However, these methods struggle with continuous action spaces, as they often assume a discrete action set, leading to suboptimal control signals and inefficient motion. Policy gradient-based methods are designed for continuous control tasks and are commonly used for UAV navigation [11,12]. While these methods generally rely on unimodal Gaussian policies, they often converge to a single action mode, neglecting other potentially superior action modes in the multimodal action space. For instance, when navigating around an obstacle, a UAV might be equally efficient going left or right, but a unimodal Gaussian policy would only learn one of these directions. This limitation reduces exploration efficiency and policy expressiveness. Model-based reinforcement learning integrates environmental models to improve sample efficiency [13,14]. Although these methods can lead to faster convergence, they are heavily dependent on model accuracy, and model biases can accumulate, leading to suboptimal performance in dynamic environments.

The primary goal of this paper is to enhance the path planning capabilities of UAVs by leveraging Group Relative Policy Optimization (GRPO) and multimodal diffusion strategies. Unlike existing diffusion-based navigation methods that primarily focus on trajectory prediction or single-mode generation, our approach uses diffusion models for multimodal action generation within a reinforcement learning framework. This enables the exploration of multiple trajectory modes from the same state. GRPO replaces the value network with intra-group trajectory ranking, in contrast to standard actor-critic methods that suffer from value estimation errors in sparse-reward environments. Despite these advantages, two challenges persist. First, the scarcity of successful samples and sparse reward signals often result in the rejection of many collision-prone trajectories, preventing valuable learning from failure experiences. Second, current methods commonly use probabilistic penalties for collision avoidance, which may not guarantee safety.

To address these challenges, we propose a novel framework for UAV autonomous path planning and obstacle avoidance based on Group Relative Policy Optimization and Multimodal Diffusion Strategy (GRPO-MDP). Specifically, we employ a diffusion-based generative model to construct a multimodal policy network, allowing for the flexible representation of multiple feasible trajectories in the same state. This enhances the policy's expressiveness and improves exploration efficiency. We combine the diffusion method for action generation with a post-trajectory relabeling mechanism to help mitigate the sparse reward problem. The Control Barrier Function (CBF) is incorporated as a real-time safety filter to enforce hard safety constraints, ensuring that UAVs do not violate safety limits, especially in high-stakes environments. The main contributions of this paper are as follows:

1. This work introduces the combination of a diffusion strategy with action generation for UAV autonomous path planning, demonstrating that iterative denoising can effectively generate feasible actions in high-dimensional action spaces.
2. This work propose GRPO-MDP, which constructs an advantage function by using intra-group trajectory ranking to generate safe, efficient, and stable paths.
3. GRPO-MDP outperforms existing methods in terms of success rate, path efficiency, and safety in dynamic environments, providing a more reliable and efficient solution for UAV autonomous navigation.

## 2. Related work

Autonomous path planning and obstacle avoidance for UAVs、 has become a prominent research focus within robotics and artificial intelligence. Various path planning approaches have been proposed in the literature, and they can be broadly categorized into sampling-based, optimization-based, imitation learning-based, and DRL-based methods.

### 2.1. Sampling-based planning methods

Sampling-based methods construct path graphs by randomly or heuristically sampling the state space, with RRT being the most widely recognized. Huang et al. [3] proposed the Adaptive Bidirectional Quick Optimal RRT algorithm, which introduces a bidirectional search strategy along with an adaptive sampling mechanism. This approach improves search efficiency while ensuring path optimality. Xu et al. [4] introduced an autonomous UAV exploration method using incremental sampling and probabilistic roadmaps for dynamic environment exploration. By maintaining a continuously updated probabilistic roadmap, UAVs can explore unknown dynamic environments efficiently, while the incremental sampling strategy reduces the overhead of repeated computation. However, in high-dimensional state spaces, the sampling efficiency of these methods decreases exponentially as the dimension increases, leading to excessive computational complexity. Furthermore, the paths generated by these methods are typically connected by discrete sampling points, which lack smoothness and often require additional post-processing for practical control applications.

### 2.2. Optimization-based methods

Optimization-based methods solve problems by simulating natural phenomena such as evolution and swarm intelligence. Liu et al. [6] proposed the A*-APF algorithm, which combines the A* algorithm for global path search in a spatiotemporal grid with an artificial potential field method for local obstacle avoidance and path smoothing. This approach utilizes a dynamic repulsion potential field to represent mobile threats. Fu et al. [14] introduced an optimizer inspired by the foraging behavior of the red-billed blue magpie (RBMO), which simulates the exploration, development, and group cooperation behaviors of birds. Lyu et al. [16] proposed the improved Marine Predator Algorithm (MMPA), which incorporates the Lévy flight strategy and adaptive search mechanism to dynamically balance local search and global exploration. For multi-task, multi-UAV cooperative planning, Hu et al. [17] developed the co-evolutionary multigroup particle swarm optimization (CMPSO) algorithm, which divides the drone population into multiple subgroups that achieve co-evolution through information sharing and competition mechanisms. Huang et al. [15] proposed the Density Gradient-RRT algorithm, which improves path quality by increasing sampling density in areas with high obstacle density. Despite their effectiveness, optimization-based methods suffer from high computational complexity and usually require the evaluation of thousands or even tens of thousands of candidate solutions. This makes them unsuitable for real-time planning in dynamic environments. Additionally, these methods are typically isolated optimization processes, unable to leverage historical experience, which limits their ability to transfer knowledge across scenarios.

### 2.3. Imitation learning-based methods

Imitation learning methods train policy models by learning from expert demonstration data, providing end-to-end solutions for UAV autonomous navigation. Chen et al. [7] proposed a method for imitation learning from imperfect demonstrations. The learning weights are dynamically adjusted according to the quality of the demonstration trajectories, enabling the extraction of useful information even from suboptimal or erroneous data. Wei et al. [8] introduced a vision-based imitation learning approach, where convolutional neural networks (CNNs) are used to predict control commands directly from RGB images. The system is trained using flight data collected from human experts in real orchard environments. However, imitation learning-based methods require substantial data collection costs, and their performance is often

limited to the level of the expert demonstrations. These methods cannot autonomously discover superior strategies through exploration and generally fail to surpass expert-level performance in novel or untrained scenarios.

### 2.4. DRL-based methods

DRL offers a more adaptive solution for UAV path planning by enabling the agent to autonomously learn decision-making strategies through interaction with the environment. Wu et al. [9] proposed an adaptive conversion speed Q-learning algorithm, which allows UAVs to dynamically balance search efficiency and path quality by adjusting the learning rate and exploration strategy. For energy consumption optimization, Zhu et al. [10] applied DRL to UAV trajectory planning in wireless sensor networks, aiming to extend the network lifetime by minimizing energy consumption. Liu et al. [11] introduced the DRL-based Unmanned Trajectory Planning System (DRL-UTPS), which optimizes trajectory planning while accounting for dynamic changes in network topology and data transmission impacts. Wei et al. [12] proposed a joint UAV trajectory planning, directed acyclic graph (DAG) task scheduling, and service function deployment method based on DRL from the perspective of edge computing. Dhuheir et al. [13] developed a DRL-based method for trajectory path planning and distributed reasoning for resource-constrained UAV clusters, using a hierarchical reinforcement learning architecture to decouple global trajectory planning from local obstacle avoidance and task deployment. Most existing approaches rely on unimodal Gaussian or deterministic policies, which fail to effectively represent multiple near-optimal trajectories in the same state. This limitation reduces exploration efficiency and the ability to generate diverse, viable solutions in dynamic environments. Additionally, DRL generally requires large amounts of interaction data to learn effective policies. Existing approaches mainly use negative penalties for collisions or failed trajectories, but these methods often overlook the valuable information contained in failed experiences. As a result, the learning process is not as efficient as it could be, and the full potential of failure experiences is not realized, limiting the overall effectiveness of DRL-based UAV path planning.

**Table 1**
Symbols and Abbreviations.

| Symbol | Description |
|---|---|
| $S$ | State space |
| $A$ | Action space |
| $P$ | State transition probability |
| $R$ | Reward function |
| $\gamma$ | Discount factor |
| $\pi_\theta$ | Policy parameterized by θ |
| $K$ | Number of diffusion steps |
| $G$ | Group size |
| $B$ | Batch size |
| $d_{safe}$ | Safety threshold distance |
| $\alpha_k$ | Noise scheduling coefficient |
| $\epsilon_\theta$ | Denoising network |
| $R_i$ | Trajectory return |
| $W_2$ | Wasserstein distance |
| $\Delta t$ | Control time step |
| **Abbreviation** | **Full Name** |
| UAV | Unmanned Aerial Vehicle |
| GRPO | Group Relative Policy Optimization |
| MDP | Markov Decision Process |
| CBF | Control Barrier Function |
| DRL | Deep Reinforcement Learning |
| DDPM | Denoising Diffusion Probabilistic Model |
| DDIM | Denoising Diffusion Implicit Model |
| RRT | Rapidly-exploring Random Tree |
| PRM | Probabilistic Roadmap |
| FPS | Farthest Point Sampling |
| QP | Quadratic Programming |
| FiLM | Feature-wise Linear Modulation |
| KL | Kullback-Leibler |
| ELBO | Evidence Lower Bound |

## 3. Nomenclature

Table 1 shows the symbols and abbreviations used in this article.

## 4. GRPO-MDP overall framework

In autonomous navigation tasks, UAVs often encounter multiple feasible trajectories under the same environmental state. Conventional unimodal strategies, such as Gaussian policies, are insufficient for capturing the multi-modal action distribution, which results in a limited representation of the state space. Consequently, successful experiences

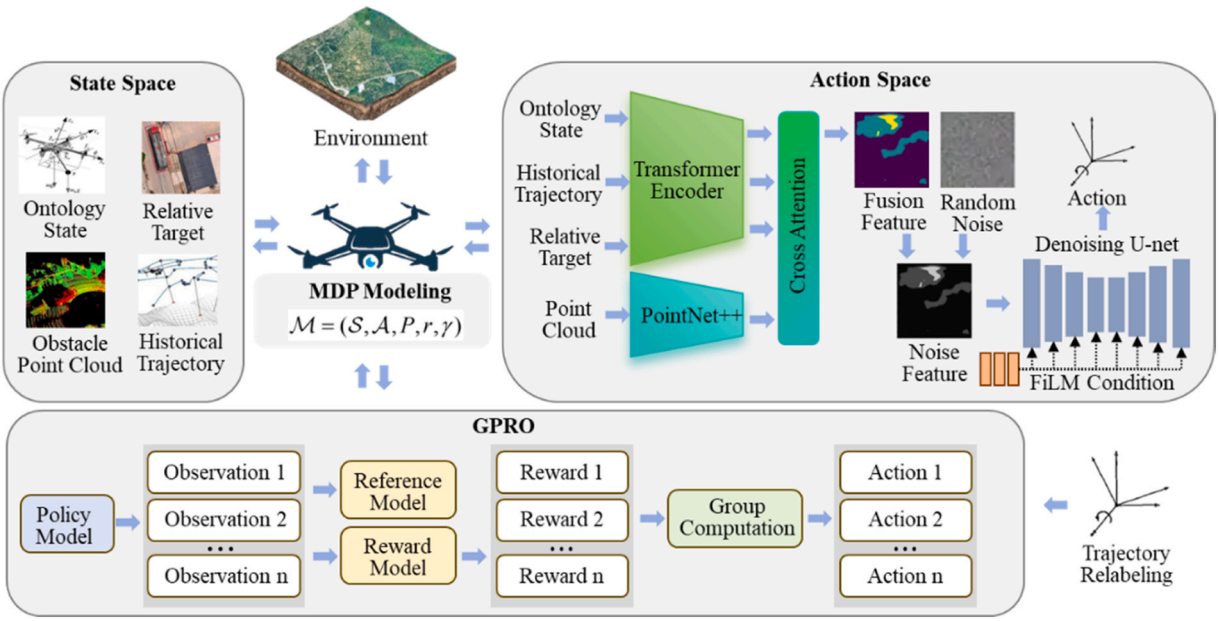



**Fig. 1.** Overall flowchart of the Group Relative Policy Optimization and Multimodal Diffusion Strategy algorithm.

become sparse, while many failure experiences remain unexploited. Additionally, collision avoidance in safety-critical scenarios must be treated as absolute, rather than being merely penalized probabilistically. To address these challenges, this paper proposes the GRPO-MDP framework (e.g. Fig. 1). The framework integrates a diffusion policy network that models the conditional multimodal action distribution using a diffusion generative model, enabling diverse action sampling during inference. As shown in Algorithm 1, the GRPO method constructs an advantage function through intra-group trajectory ranking, thereby eliminating the need for value networks and improving stability. It also introduces a hindsight trajectory relabeling mechanism that leverages post-hoc information from failed trajectories to relabel goals, transforming these failed experiences into effective learning signals. GRPO's computational overhead consists of three components. The trajectory collection computational cost is $O(B{\cdot}T{\cdot}d)$, where $B$ is the batch size, $T$ is the trajectory length, and $d$ is the action dimension. The cost for k-means with I iterations is $O(B{\cdot}G{\cdot}I)$, and the cost for Z-score normalization within groups is $O(B)$.

**Algorithm 1. GRPO-MDP Algorithm for UAV Path Planning and Obstacle Avoidance**

**Input:** Initial policy $\pi_\theta$, diffusion steps $K$, group size $G$, batch size $B$, safety threshold $d_{\text{safe}}$

**Output:** Trained policy $\pi_{\theta^*}$

**for** each training iteration **do**
  Initialize $\mathcal{D}_{\text{batch}} = \varnothing$
  for each episode $e = 1$ to $B$ do
    Reset environment, get initial state $s_0 = [s_0^{\text{ego}}, s_0^{\text{goal}}, s_0^{\text{obs}}, s_0^{\text{context}}]$
    Initialize trajectory $\tau = \{(s_0, a_0)\}$
    **for** each time step $t$ **do**
      Encode obstacle point cloud $h_{\text{obs}} = \text{MaxPool}(\text{MLP}_{\text{global}}(\text{EdgeConv}(s_t^{\text{obs}})))$
      Encode temporal context $s_t^{\text{context}}$
      Fuse features $h_{\text{fuse}} = \text{CrossAttention}([h_{\text{ego}}, h_{\text{obs}}, h_{\text{context}}])$
      Sample Gaussian noise $a_t^{(K)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$
      **for** diffusion step $k = K$ to $1$ **do**
        Compute noise schedule $\overline{\alpha}_k$
        Denoise action $a_t^{(k-1)}$
      **end for**
      Compute barrier function $h(s_t) = \min_{o_j} d(\mathbf{p}, \mathbf{o}_j) - d_{\text{safe}}$
      **if** $\nabla h(s_t) \cdot f(s_t, a_t) < -\frac{\gamma_{\text{cbf}}}{\Delta t} h(s_t)$ **then**
        Compute $a_{\text{safe}} = \arg\min_a \| a - a_t \|^2$ subject to CBF constraint
        Compute CBF penalty $r_t^{\text{cbf}} = -\alpha_{\text{cbf}} \cdot \max(0, -\dot{h}(s_t, a_t) - \gamma_{\text{cbf}} h(s_t))^2$
      **else**
        $r_t^{\text{cbf}} = 0$
      Execute action $a_t$, observe $s_{t+1}$, receive reward $r_t^{\text{task}}$
      Compute total reward $r_t = r_t^{\text{task}} + r_t^{\text{cbf}} + r_t^{\text{smoothness}}$
      Add transition $(s_t, a_t, r_t, s_{t+1})$ to trajectory $\tau$
    **end for**
    Compute $\tau_{\text{original}} = (\tau, g_{\text{original}})$ with original goal $\mathbf{p}_{\text{goal}}$
    if collision occurred at $t_c$
      Set implicit target $\mathbf{p}_* = \mathbf{p}_{t_c}$, Relabel reward $r_t^{\text{new}}$
      Apply Gaussian penalty $r_t^{\text{new}}$ for $t < t_c$
    Add $\tau_{\text{original}}$ and $\tau_{\text{relabeled}}$ (if exists) to $\mathcal{D}_{\text{batch}}$
  **end for**
  Adapt group size based on difficulty
  **for** each trajectory $\tau$ in $\mathcal{D}_{\text{batch}}$ **do**
    Compute trajectory return $R(\tau)$ and importance weight $w(\tau)$
  **end for**
  Group trajectories by similarity and compute advantages and compute GRPO objective $\mathcal{L}_{\text{grpo}}$
  Update diffusion policy $\theta$ and update denoising network via score matching
**end for**

### 4.1. Design of state and action space

The UAV navigation task is formulated as a Markov Decision Process (MDP) [18] defined by $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P$ is the state transition probability, $r$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. At time step $t$, the state $s_t \in \mathcal{S}$ consists of four subspaces $s_t = [s_t^{\text{ego}}, s_t^{\text{goal}}, s_t^{\text{obs}}, s_t^{\text{context}}]$. The ego-state $s_t^{\text{ego}}$ represents the instantaneous kinematic and dynamic state of the UAV is $s_t^{\text{ego}} = [\mathbf{p}_t, \mathbf{v}_t, \mathbf{q}_t, \boldsymbol{\omega}_t]$, where $\mathbf{p}_t$ is the 3D position vector of the UAV, $\mathbf{v}_t$ is the 3D velocity vector, $\mathbf{q}_t$ is the attitude quaternion, ensuring continuous representation without gimbal lock, and $\boldsymbol{\omega}_t$ is the angular velocity vector. The goal-relative subspace $s_t^{\text{goal}}$ encodes the position and orientation of the UAV relative to its target:

$$s_t^{\text{goal}} = [\mathbf{p}_{\text{goal}} - \mathbf{p}_t, \left\| \mathbf{p}_{\text{goal}} - \mathbf{p}_t \right\|_2, \theta_{\text{goal}}] \tag{1}$$

where $\mathbf{p}_{\text{goal}}$ is the target position, $\left\| \mathbf{p}_{\text{goal}} - \mathbf{p}_t \right\|_2$ is the Euclidean distance [19] to the target, and $\theta_{\text{goal}}$ is the azimuth angle between the UAV

heading and target direction, enhancing directional awareness. The obstacle observation subspace $s_t^{\text{obs}}$ encodes the local obstacle point cloud and their relative velocities is $s_t^{\text{obs}} = \{(\mathbf{p}_{\text{rel},i}, \mathbf{v}_{\text{rel},i})\}_{i=1}^{N_{\text{obs}}}$, where $\mathbf{p}_{\text{rel},i} = \mathbf{p}_{o,i} - \mathbf{p}_t$ is the relative position of obstacle $i$, $\mathbf{v}_{\text{rel},i} = \mathbf{v}_{o,i} - \mathbf{v}_t$ is the relative velocity, and $N_{\text{obs}}$ is the number of obstacles within the sensing range. Each obstacle feature consists of a 3D relative position vector $(o_{i,x}^{\text{rel}}, o_{i,y}^{\text{rel}}, o_{i,z}^{\text{rel}})$ and a 3D relative velocity vector $(v_{i,x}^{\text{rel}}, v_{i,y}^{\text{rel}}, v_{i,z}^{\text{rel}})$, yielding a 6-dimensional feature per obstacle. The point cloud input supports up to $N_{\max} = 100$ obstacles; when fewer obstacles are present, zero-padding is applied, and when more obstacles are detected, only the nearest 100 are retained based on Euclidean distance to the UAV. To capture temporal dependencies and motion continuity, the historical context $s_t^{\text{context}}$ is encoded using a Transformer. Specifically, the state-action pairs of the past $T$ steps are encoded as:

$$s_t^{\text{context}} = \text{Enc}_{\text{Transformer}}(\{(s_{t-T}, a_{t-T}), \ldots, (s_{t-1}, a_{t-1})\}) \tag{2}$$

where $T = 10$ is the temporal window length, and $\text{Enc}_{\text{Transformer}}(\cdot)$ denotes a Transformer encoder that compresses the history into a latent representation, enabling the policy to model memory and periodic behaviors. The control action $a_t \in \mathscr{A}$ represents the commanded acceleration $a_t = [a_x, a_y, a_z]^\top$, subject to the constraint $a_t \in [-a_{\max}, a_{\max}]^3$, where $a_x, a_y, a_z$ are the acceleration commands along the UAV body axes, and $a_{\max} = 3\text{m/s}^2$ is the maximum allowable acceleration, determined by UAV hardware limitations. Since acceleration is the time derivative of velocity, this control mode inherently ensures smooth trajectories, reducing mechanical stress and energy consumption.

### 4.2. Design of diffusion strategy network

Multiple equivalent or near-optimal trajectories may exist for a UAV under the same state. A diffusion-based strategy network is proposed, which models the conditional action distribution $p(a_t|s_t)$ as a flexible implicit generative process capable of expressing arbitrary trajectory distributions. Given a ground-truth action $a_t^{(0)}$, the forward diffusion process gradually corrupts the action signal by adding Gaussian noise over $K$ steps using a Markov chain [20]:

$$a_t^{(k)} = \sqrt{\alpha_k}\, a_t^{(0)} + \sqrt{1-\alpha_k}\,\epsilon_k, \quad \epsilon_k \sim \mathscr{N}(0, I) \tag{3}$$

where $k \in \{1, 2, \ldots, K\}$ is the diffusion step index and $\alpha_k \in (0, 1)$ is the noise scheduling coefficient. The noise scheduling coefficient $\alpha_k$ follows a linear schedule:

$$\beta_k = \beta_{\text{start}} + \frac{k}{K}(\beta_{\text{end}} - \beta_{\text{start}}), \quad \alpha_k = 1 - \beta_k, \quad \overline{\alpha}_k = \prod_{i=1}^{k} \alpha_i \tag{4}$$

where $\beta_{\text{start}} = 10^{-4}$ and $\beta_{\text{end}} = 0.02$ are chosen based on the action space bounds $[-a_{\max}, a_{\max}]^3$. The linear schedule provides a gradual noise injection that matches the scale of acceleration commands, ensuring that early diffusion steps capture coarse trajectory structure while later steps refine control adjustments. At inference time, starting from Gaussian noise $a_t^{(K)} \sim \mathscr{N}(0, I)$, the action is reconstructed iteratively through a learned denoising network $\epsilon_\theta$ as:

$$a_t^{(k-1)} = \frac{1}{\sqrt{\alpha_k}}\left[a_t^{(k)} - \frac{1-\alpha_k}{\sqrt{1-\overline{\alpha}_k}}\,\epsilon_\theta\left(a_t^{(k)}, k, s_t\right)\right] + \sigma_k z \tag{5}$$

where $\overline{\alpha}_k = \prod_{i=1}^{k} \alpha_i$ is the cumulative noise coefficient, $\sigma_k$ is the sampling variance controlling generation diversity, and $z \sim \mathscr{N}(0, I)$ is the reparametrized noise vector. The denoising function $\epsilon_\theta(a_t^{(k)}, k, s_t)$ is parameterized by a U-Net backbone. The network simultaneously encodes noisy actions, diffusion step indices, and heterogeneous UAV state information. State encoders are designed for different subspaces of $s_t$. The UAV's instantaneous kinematic features are captured by a Transformer encoder. Temporal dependencies and motion continuity, represented by $h_{\text{context}}$, are also modeled using a Transformer encoder [21]. The past state-action sequence is projected into tokens using sinusoidal positional encoding. The output corresponding to the most recent timestep summarizes the historical evolution, capturing long-range temporal correlations. The obstacle point cloud is processed using a PointNet++ [22] backbone with hierarchical sampling and grouping:

$$h_{\text{obs}} = \text{MaxPool}(\text{MLP}_{\text{global}}(\text{EdgeConv}(s_t^{\text{obs}}))) \tag{6}$$

where $\text{EdgeConv}(\cdot)$ constructs a $k$-nearest neighbor graph to model local geometric relationships between points. Each point feature consists of the relative position $\mathbf{p}_{\text{rel}}$ and relative velocity $\mathbf{v}_{\text{rel}}$, enabling dynamic obstacle motion inference. The point set is down sampled layer-by-layer using Farthest Point Sampling (FPS) [23], progressively enlarging the receptive field. Finally, global max pooling extracts a global descriptor that is invariant to input order. The heterogeneous features are fused using a cross-attention mechanism:

$$h_{\text{state}} = \text{CrossAttention}([h_{\text{ego}}, h_{\text{obs}}, h_{\text{context}}]) \tag{7}$$

where $h_{\text{ego}}$ serves as the query, and $[h_{\text{obs}}, h_{\text{context}}]$ provide the key-value pairs. This allows the policy to selectively attend to relevant obstacle patterns and temporal dynamics based on the UAV's current motion state. The fused state representation $h_{\text{state}}$ is injected into each U-Net layer via a FiLM (Feature-wise Linear Modulation) mechanism [24]. In each residual block, FiLM modulates the intermediate features:

$$h_{\text{modulated}} = \gamma(k, h_{\text{state}}) \odot h + \beta(k, h_{\text{state}}) \tag{8}$$

where $\gamma$ and $\beta$ are learned affine parameters, and $\odot$ represents element-wise multiplication. The multi-scale encoder–decoder structure of the U-Net allows for the simultaneous modeling of global motion trends and fine-grained local adjustments, satisfying the smoothness requirements of continuous UAV control. The diffusion policy learns the conditional distribution $p(a_t|s_t)$ via denoising score matching:

$$\mathscr{L}_{diffusion} = \mathbb{E}_{k\sim u\left(1,K\right),\ \int\sim\mathscr{N}\left(0,I\right)}\left[\left\| \int - \int_\theta \left(a_t^{(k)}, k, s_t\right)\right\|_2^2\right] \tag{9}$$

which is equivalent to maximizing the variational lower bound (ELBO) and minimizing the Kullback–Leibler divergence [25] between the true and model distributions. By uniformly sampling diffusion steps $k$, the network learns to denoise actions under varying noise levels, ensuring robust reconstruction across scales. During inference, the process begins with Gaussian noise $a_t^{(K)} \sim \mathscr{N}(0, I)$. Multiple independent samplings produce a candidate action set $\{a_t^{(i)}\}_{i=1}^{M}, \quad M = 8$, where each sample corresponds to a different mode in the multimodal action distribution.

### 4.3. Design of group relative policy optimization

The GRPO [26] framework builds advantages by ranking trajectories within a group. It addresses the challenge of assigning absolute values to actions and instead focuses on relative performance. Specifically, for a set of $G$ trajectories $\{\tau_i\}_{i=1}^{G}$ collected under the current policy $\pi_\theta$. Trajectories are grouped based on a composite similarity metric that considers both initial conditions and outcome characteristics. Formally, for trajectories $\tau_i$ and $\tau_j$, the similarity score is computed as:

$$\text{sim}\left(\tau_i, \tau_j\right) = \exp\left(-\frac{\left\|s_0^i - s_0^j\right\|^2}{2\sigma_s^2}\right) \cdot \exp\left(-\frac{\left|g_i - g_j\right|^2}{2\sigma_g^2}\right) \tag{10}$$

where $s_0^i, s_0^j$ are the initial states, $g_i, g_j$ are the goal positions, $\sigma_s = 2.0\text{m}$ and $\sigma_g = 3.0\text{m}$ are bandwidth parameters. Trajectories are then clustered using k-means with $k = B/G$ clusters, where $B$ is the batch size and

$G$ is the target group size. Within each cluster, Z-score normalization is applied independently. This ensures that trajectories facing similar initial challenges are compared fairly, preventing advantageous trajectories from trivially easy scenarios from dominating the advantage calculation. K-means clustering has a computational cost of $\mathbf{O}(\mathbf{nkI})$, where $\mathbf{I}$ represents the number of iterations. Hierarchical clustering, on the other hand, incurs approximately 4 times the computational overhead. DBSCAN, however, failed to produce consistent group sizes, which are required for stable Z-score normalization. The relative advantage of each trajectory is calculated using Z-score normalization:

$$\widehat{A}_i = \frac{R(\tau_i) - \mu_G}{\sigma_G + \epsilon} \tag{11}$$

where $R(\tau_i)$ represents the trajectory reward:

$$R(\tau_i) = \sum_{t=0}^{T} \gamma^t r_t^{(i)} \tag{12}$$

with $\gamma = 0.99$ being the discount factor. Here, $\mu_G$ and $\sigma_G$ are the mean and standard deviation of the rewards within the group. The GRPO objective function:

$$\mathscr{L}_{\text{GRPO}} = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \widehat{A}(\tau)\nabla_\theta \log \pi_\theta\left(a_t|s_t\right)\right] \tag{13}$$

where the advantage $\widehat{A}(\tau)$ enhances the probability of high-reward trajectories and reduces the likelihood of low-reward trajectories. To prevent overly aggressive policy updates, a KL divergence penalty is incorporated:

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{GRPO}} - \beta\mathbb{E}_s[D_{\text{KL}}(\pi_{\text{old}}(\cdot|s) \parallel \pi_\theta(\cdot|s))] \tag{14}$$

where $D_{\text{KL}}$ is the Kullback-Leibler divergence between the old and new policy distributions. For diffusion policies, which are implicitly distributed, calculating the KL divergence is not straightforward. Therefore, we approximate the KL divergence using the Wasserstein distance $D_W$. The Wasserstein distance measures the minimum transport cost between two distributions and can be computed as:

$$D_W(\pi_{\text{old}}, \pi_\theta) \approx \mathbb{E}_s[\|a_{\text{old}}(s) - a_\theta(s)\|_2] \tag{15}$$

where $a_{\text{old}}(s)$ and $a_\theta(s)$ are the actions generated by deterministic sampling from the old and new policies, respectively, at state $s$ using Denoising Diffusion Implicit Models. This approximation allows the policy to remain stable while exploring the action space. Unlike value-based methods, GRPO's relative comparison operates solely on scalar trajectory returns, making it dimension-agnostic. The reward distributions of different environments can vary. In simpler scenarios, where most trajectories yield similar rewards, the variance $\sigma_G$ approaches zero, leading to an advantage in the Z-score normalization. Conversely, in more difficult environments with broader reward distributions, the variance is larger, requiring a more careful estimation of statistics. To adapt to this variability, a scene difficulty metric $D(s_0)$ is introduced, which adjusts the group size $G$ based on the environment's characteristics. The difficulty metric is based on two factors: obstacle density and the distance to the target:

$$D(s_0) = w_d \cdot \frac{N_{\text{obs}}}{V_{\text{sense}}} + w_{\text{dist}} \cdot \left\|\mathbf{p}_0 - \mathbf{p}_{\text{goal}}\right\|_2 \tag{16}$$

where $N_{\text{obs}}$ is the number of obstacles within the perception range, $V_{\text{sense}}$ is the volume of the UAV's sensing range, and $w_d = 0.7$ and $w_{\text{dist}} = 0.3$ are weights reflecting the relative importance of obstacle density and target proximity. Based on this difficulty metric, the group size $G$ is adapted according to the following rules:

$$G = \begin{cases} 8 & \text{if} D > D_{\text{high}} \\ 16 & \text{if} D_{\text{mid}} < D \le D_{\text{high}} \\ 32 & \text{if} D \le D_{\text{mid}} \end{cases} \tag{17}$$

where $D_{\text{high}}$ and $D_{\text{mid}}$ are predefined thresholds for high and medium difficulty scenarios. GRPO is fundamentally an on-policy algorithm, meaning it relies on data collected from the current policy $\pi_\theta$. However, during the early stages of training, when policy updates are slow, discarding all previously collected data would result in inefficient learning. To address this issue, a correction weight is introduced for trajectories $\tau$ collected under the old policy $\pi_{\text{old}}$. The weight is given by:

$$w(\tau) = \frac{\prod_{t=0}^{T} \pi_\theta(a_t|s_t)}{\prod_{t=0}^{T} \pi_{\text{old}}(a_t|s_t)} \tag{18}$$

This correction ensures that previously collected trajectories are appropriately weighted, preventing the waste of valuable experience and enabling smoother policy transitions between iterations. By incorporating old data into the training process, the algorithm accelerates convergence without sacrificing stability.

### 4.4. Design of hindsight trajectory relabeling mechanism

In sparse reward environments, agents must often explore extensively in order to trigger successful outcomes. Many trajectories that initially appear to be failures may actually be valuable when evaluated under different objectives. This paper introduces an innovative dual-mode Hindsight Relabeling Mechanism [27], consisting of the virtual target mode and the safe boundary mode. Virtual target mode is applied to trajectories that fail to reach the original goal $g$ within the episode time limit but do not result in any collision. Safety boundary mode is applied to all trajectories that result in a collision. In virtual target mode, the relabeled goal $g^{'}$ is set to the final position $p_T$ reached before timeout. No additional optimization or trajectory selection is performed in this process in order to avoid introducing bias toward specific trajectory shapes. In safety boundary mode, the safety boundary is implicitly defined by the collision point $p_{col}$ and a Gaussian kernel with bandwidth $\sigma_{col} = 1.5m$. No explicit boundary surface is computed; instead, a penalty field smoothly attenuates outward from the collision point. The relabeling process introduces approximately 12 % additional computational overhead per training iteration, which is offset by a 2.1 × improvement in sample efficiency. Since relabeled trajectories may receive higher rewards than under the original goal distribution, a 50/50 mixing ratio between original and relabeled data is used. This ensures that the policy maintains fidelity to the true task objective while benefiting from auxiliary supervision. The reward for such a trajectory is then recalculated as:

$$r_t^{\text{new}} = -\|\mathbf{p}_t - \mathbf{p}_T\| + 1(t = T)\cdot R_{\text{reach}} \tag{19}$$

where $\mathbf{p}_t$ is the current position and $\mathbf{p}_T$ is the implicit target. $R_{\text{reach}} = 50$ is the reward for reaching the implicit target. The goal in the state representation $s_t^{\text{goal}}$ is updated by replacing $\mathbf{p}_{\text{goal}}$ with $\mathbf{p}_T$ to ensure consistency between the state, action, and reward. This adjustment allows the model to refine its understanding of what constitutes a successful outcome, even when the explicit goal is not met. In the safe boundary mode, a trajectory that results in a collision at time $t_f$ is relabeled by adjusting the reward to reinforce safety awareness. Although the pre-collision decision may not have directly caused the collision, it could have led the UAV into a danger zone. To reinforce safe behavior, a penalty is applied after the collision and backpropagated using a Gaussian kernel. The relabeled reward for a trajectory with a collision at time $t_f$ is given by:

$$r_t^{\text{new}} = r_t^{\text{original}} - \lambda \exp\left(-\frac{\left\|\mathbf{p}_t - \mathbf{p}_{t_f}\right\|^2}{2\sigma^2}\right), \quad t < t_f \tag{20}$$

where $r_t^{\text{original}}$ is the original reward at time step $t$, and $\mathbf{p}_{t_f}$ is the position of the UAV at the time of collision. $\lambda = 30$ is a penalty intensity parameter, and $\sigma = 2$ meters determine the impact range of the penalty, with stronger penalties within 1 m of the collision point and minimal impact beyond 3 m. The relabeling is only performed for timesteps before the collision occurs ($t < t_f$), and once a collision is detected, the state is marked as terminated. This ensures that the agent learns to avoid such dangerous situations in future episodes. The use of a Gaussian kernel ensures that the penalty smoothly diminishes as the UAV moves farther from the collision point, preventing abrupt penalties for minor violations. To combine both original and relabeled trajectories, each training batch $D_{\text{train}}$ consists of two sets:

$$D_{\text{train}} = \left\{\left(\tau_{\text{original}}, g_{\text{original}}\right)\right\} \cup \left\{\left(\tau_{\text{relabeled}}, g_{\text{virtual}}\right)\right\} \tag{21}$$

where $(\tau_{\text{original}}, g_{\text{original}})$ represents the original trajectory with its associated goal, and $(\tau_{\text{relabeled}}, g_{\text{virtual}})$ represents the relabeled trajectory with a modified reward function. Each batch consists of 50 % original trajectories and 50 % relabeled trajectories. The original trajectories retain the true distribution of task objectives, while the relabeled trajectories offer auxiliary supervision to improve exploration and safety awareness during training.

### 4.5. Design of safety constraints and collision avoidance reinforcement

During the optimization process, an agent may inadvertently learn risky strategies, where it sacrifices a low-probability collision for a significantly shorter path, yielding a higher expected return. Such behavior is unacceptable in UAV navigation for manned drones or medical robots, where a single failure could lead to catastrophic consequences. To ensure the safety of the agent's actions, a CBF [28] is employed as a real-time safety filter, embedded directly into the policy training process through soft CBF penalties. The safety state space is defined as $C = \{s | h(s) \geq 0\}$, where $h(s)$ is the barrier function, which characterizes the safety boundary. For UAV obstacle avoidance, the barrier function is defined as:

$$h(s) = \min_{i \in O} d(\mathbf{p}, \mathbf{o}_i) - d_{\text{safe}} \tag{22}$$

where $d(\mathbf{p}, \mathbf{o}_i)$ is the Euclidean distance from the UAV position $\mathbf{p}$ to the $i$-th obstacle $\mathbf{o}_i$, $d_{\text{safe}} = 0.5$ meters is the safety threshold, and $\min_{i \in O}$ selects the nearest obstacle to ensure global safety. The value of $h(s)$ indicates the safety margin. The forward invariance of the safety set $C$ is guaranteed, meaning that once the UAV is inside the safe zone, it will remain there under any admissible control. For a continuous-time system where $\dot{s} = f(s, a)$, the action $a$ must satisfy the CBF condition:

$$\dot{h}(s, a) = \nabla h(s) \cdot f(s, a) \geq -\gamma_{\text{cbf}} h(s) \tag{23}$$

where $\nabla h(s)$ is the gradient of the barrier function, $\gamma_{\text{cbf}} > 0$ is a positive constant, and $\dot{h}(s, a)$ is the time derivative of the barrier function. Since the UAV control is discrete (50 Hz), the continuous-time CBF condition is discretized using a first-order Euler approximation $h(s_{t+1}) = h(s_t) + \Delta t \cdot \dot{h}(s_t, a_t)$, leading to the discrete-time CBF condition:

$$h(s_t) + \Delta t \cdot \nabla h(s_t) \cdot f(s_t, a_t) \geq (1 - \gamma_{\text{cbf}} \Delta t) h(s_t) \tag{24}$$

Simplifying the condition, we obtain:

$$\nabla h(s_t) \cdot f(s_t, a_t) \geq -\frac{\gamma_{\text{cbf}}}{\Delta t} h(s_t) \tag{25}$$

where $\Delta t = 0.02$ s and $\gamma_{\text{cbf}} = 0.1$, yielding an effective constraint rate $\gamma_{\text{eff}} = 5$. If the action $a_\pi$ produced by the diffusion policy violates the CBF condition, the nearest safe action is obtained through quadratic programming (QP):

$$a_{\text{safe}} = \text{argmin}_a, a - a_\pi,^2, \quad \text{subjectto}: \quad \nabla h(s_t) \cdot f(s_t, a) \geq -\gamma_{\text{eff}} h(s_t) \tag{26}$$

For acceleration-controlled UAVs, the system dynamics are defined as $f(s_t, a_t) = [v_t, a_t, 0, 0]^T$, where $v_t$ is the velocity and $a_t$ is the acceleration. The gradient $\nabla h(s_t)$ is computed as:

$$\nabla h(s_t) = \nabla_{\mathbf{p}} \left[\min_i d(\mathbf{p}, \mathbf{o}_i) - d_{\text{safe}}\right] = \frac{\mathbf{p} - \mathbf{o}_{i^*}}{\|\mathbf{p} - \mathbf{o}_{i^*}\|} \tag{27}$$

where $i^* = \text{argmin}_i d(\mathbf{p}, \mathbf{o}_i)$ denotes the index of the nearest obstacle. To encourage the policy to actively learn safe actions during training, a soft CBF penalty is introduced. This penalty discourages actions that violate the safety constraints:

$$r_t^{\text{cbf}} = -\alpha_{\text{cbf}} \cdot \max\left(0, -\dot{h}\left(s_t, a_t\right) - \gamma_{\text{cbf}} h(s_t)\right)^2 \tag{28}$$

where $\alpha_{\text{cbf}} = 5$ controls the strength of the penalty, and $\dot{h}(s_t, a_t)$ represents the violation of the safety condition. The soft penalty allows for minor violations, encouraging exploration while still penalizing significant safety violations. The CBF constraint is evaluated at every control timestep (50 Hz). The total reward at time step $t$ is defined as a weighted sum of task performance, safety, and control smoothness:

$$r_{\text{total}}(t) = r_{\text{task}}(t) + r_{\text{CBF}}(t) + r_{\text{smooth}}(t) \tag{29}$$

Where $w_{\text{task}}$, $w_{\text{CBF}}$, and $w_{\text{smooth}}$ denote the weights associated with task completion, CBF safety, and action smoothness, respectively. The task-related reward is composed of three components:

$$r_{\text{task}}(t) = r_{\text{progress}}(t) + r_{\text{goal}}(t) + r_{\text{collision}}(t) \tag{30}$$

The progress reward encourages movement toward the goal and is defined as $r_{\text{progress}}(t) = 0.1(d_{t-1} - d_t)$, where $d_t = \|p_t - g\|$ is the Euclidean distance between the UAV position $p_t$ and the goal position $g$. The goal-reaching reward is given by:

$$r_{\text{goal}}(t) = \begin{cases} 100, & \text{if} \|p_t - g\| < 0.5, \\ 0, & \text{otherwise}, \end{cases} \tag{31}$$

which provides a strong terminal incentive when the UAV reaches a tolerance radius of \$0.5\$~m around the goal. Collisions are penalized as:

$$r_{\text{collision}}(t) = \begin{cases} -100, & \text{ifacollisionisdetected}, \\ 0, & \text{otherwise}. \end{cases} \tag{32}$$

To enforce safety constraints during training, a control barrier function penalty is incorporated:

$$r_{\text{CBF}}(t) = -\lambda_{\text{CBF}} \max\left(0, -\left(\dot{h}(t) + \kappa h(t)\right)\right) \tag{33}$$

where $h(t)$ denotes the control barrier function encoding the signed safety margin with respect to obstacles, $\dot{h}(t)$is its time derivative, $\kappa > 0$ is a design parameter controlling the barrier response, and $\lambda_{\text{CBF}} = 10.0$ scales the penalty magnitude. This term penalizes violations of the CBF condition$\dot{h}(t) + \kappa h(t) \geq 0$. To promote smooth and dynamically feasible control, an action smoothness penalty is defined as:

$$r_{\text{smooth}}(t) = -\lambda_{\text{smooth}} \|a_t - a_{t-1}\|^2 \tag{34}$$

where $a_t$ and $a_{t-1}$ denote the control actions, respectively, and $\lambda_{\text{smooth}} = 0.01$ controls the strength of the smoothness regularization. No environment-specific coefficient tuning is performed; adaptation to task difficulty is handled exclusively through the group size mechanism in GRPO rather than by modifying the reward formulation.

## 5. Experimental

### 5.1. Experiment setting

#### 5.1.1. Dataset

The experiments are conducted using an AirSim-based 3D simulation environment. Three distinct environmental configurations are used to evaluate the proposed method: the Easy, Medium, and Hard environments. In the Easy environment, 5–10 static obstacles are randomly deployed within a 20 m × 20 m × 10 m space, with obstacle radii ranging from 0.3 to 0.8 m. The straight-line distance between the start and target points is between 8 and 12 m. The Medium environment contains 10–20 mixed obstacles deployed within a 30 m × 30 m × 15 m space, with 70 % static and 30 % dynamic obstacles, where the dynamic obstacles have speeds ranging from 0.5 to 2 m/s. The obstacle radii range from 0.3 to 1.2 m, and the start-target distance is between 15 and 20 m. The Hard environment consists of 20–35 dense obstacles deployed within a 40 m × 40 m × 20 m space, with 50 % static and 50 % dynamic obstacles, where the dynamic obstacles have speeds between 1 and 3 m/s. Some obstacles create narrow passages (1.5–2.5 m wide), and the start-target distance ranges from 25 to 35 m. Dynamic obstacles follow stochastic motion patterns modeled as $v_{\text{obs}}(t+1) = v_{\text{obs}}(t) + \mathcal{N}(0, \sigma_v^2) \cdot \Delta t$ with velocity bounds $v_{\text{obs}} \in [v_{\min}, v_{\max}]$ and direction changes occurring with probability $p_{\text{turn}} = 0.05$ per timestep. This stochastic model creates realistic unpredictable obstacle behavior, more challenging than deterministic sinusoidal or circular patterns used in some prior work. In each environment, 1000 random scenarios are generated for training, 200 for validation, and 500 for testing. The UAV's perception range is modeled as a spherical area with a radius of 8 m, a sensor frequency of 20 Hz, and a control frequency of 50 Hz.

#### 5.1.2. Experimental configuration

The diffusion strategy network is based on a U-Net encoder with 4 layers and channels [64,128,256,512]. The diffusion process involves 20 steps during training and 10 steps during inference, with accelerated inference using DDIM. A linear noise scheduling is applied, with noise parameters $\beta_1 = 0.0001$ and $\beta_T = 0.02$. The transformer encoder consists of 4 layers, 8 attention heads, and a hidden dimension of 256, while PointNet++ uses 512, 256, and 128 sampling points. The history window length is set to 5 steps. The GRPO optimizer uses Adam with a learning rate of $3 \times 10^{-4}$, a batch size of 256 trajectories, and a group size of 32 for easy, 64 for medium, and 128 for hard environments. The learning rate decay follows a cosine annealing schedule, and the KL divergence penalty coefficient is set to $\beta_{\text{kl}} = 0.01$. The Wasserstein distance approximation is performed with 20 samples. For the CBF, the safety threshold is $d_{\text{safe}} = 1.0$ meters, and the soft penalty coefficient is set to $\lambda_{\text{cbf}} = 10.0$. The control frequency is 50 Hz, and the smoothness penalty coefficient is $\lambda_{\text{smooth}} = 0.1$. Remarking parameters for the trajectories include virtual target and safe boundary mode weights of 0.5, a collision penalty intensity of 50.0, and a Gaussian kernel bandwidth of $\sigma = 2.0$ meters. Training is conducted over 5000 rounds, with each round involving 2048 environment interaction steps. Policy updates occur 10 times per round, using 32 parallel environments. The training dataset consists of a mix of 50 % original and 50 % relabeled trajectories, each round using a batch size of 256. The CPU is an Intel Xeon Gold 6248 R, paired with 2 NVIDIA RTX 4090 GPUs and 256 GB of DDR4 memory. The storage consists of a 2TB NVMe SSD. The operating system is Ubuntu 22.04 LTS, with Python version 3.10.12 and PyTorch version 2.1.0. The simulation platform used is AirSim 1.8.1.

#### 5.1.3. Comparison methods

For comparison, the proposed method is evaluated against a variety of state-of-the-art approaches: RRT [3], a bidirectional fast exploration random tree algorithm; PRM [4], a dynamic environment exploration method using incremental sampling and probabilistic roadmaps; A-APF* [6], a spatiotemporal grid optimization algorithm combining A* global path search and artificial potential field local obstacle avoidance; RBMO [14], a metaheuristic optimizer inspired by the foraging behavior of the red-billed blue magpie; MMPA [16], an improved marine predator algorithm integrating Lévy flight strategy and adaptive search mechanism; CMPSO [17], a co-evolutionary multi-population particle swarm optimization algorithm; Vision-IL [8], a vision-based imitation learning method; DRL-Energy [10], a DRL trajectory planning method minimizing energy consumption; DRL-UTPS [11], a DRL trajectory planning system considering network topology and data transmission effects; HDRL [13], a hierarchical reinforcement learning architecture that decouples global trajectory planning from local obstacle avoidance; Error-RL [18], a method learning to construct UAV path planning solutions with position error correction; and PointCloud-DRL [22], a multi-UAV end-to-end distributed trajectory generation method based on point cloud data.

#### 5.1.4. Evaluation metrics

To assess the effectiveness of the proposed method, several evaluation metrics are used: Success Rate (SR) is the proportion of UAVs that reach the target point within 0.5 m without collision. Path Efficiency (PE) is the ratio of the straight-line distance from the start to the target point to the actual flight path length. Average Arrival Time (AAT) is the average number of time steps required for a trajectory to successfully reach the target. Collision Rate (CR) is the proportion of trajectories that result in collisions compared to the total number of trajectories. Average Safety Margin (ASM) measures the average margin by which the UAV's distance from the nearest obstacle exceeds a safety threshold. Finally, Path Smoothness (PS) is the average value of acceleration changes; a lower PS indicates smoother trajectories and reduced energy consumption.

### 5.2. Result and Analysis

As shown in Table 1 and Fig. 2, GRPO-MDP achieved the best performance across all environments. In the Easy environment, its success rate reached 97.6 %, improving by 1.9 % over PointCloud-DRL; in the Medium environment, the success rate was 94.8 %, a 4.5 % gain; and in the Hard environment, the success rate improved by 7.3 %. This demonstrates that GRPO-MDP's advantage becomes more pronounced as the environment grows more complex. The diffusion model, through iterative denoising, generates diverse action candidates under identical conditions, enabling exploration of multiple trajectory modalities. This capability is critical in hard environments, where narrow passages and dense obstacles yield multimodal feasible path spaces.

The data show that GRPO-MDP's advantage increases from 1.9 % to 7.3 % with environmental difficulty, confirming its strong exploration ability. GRPO constructs a relative advantage function via within-group trajectory ranking, eliminating the value network and avoiding estimation errors. In complex environments with high reward variance, value networks often overfit or underfit, whereas GRPO's Z-score normalization adaptively stabilizes training by amplifying relative differences in simple cases and preventing extreme gradients in difficult ones. Experiments confirm that GRPO-MDP maintains stable training curves in the Hard environment, while other DRL methods exhibit oscillations. Sparse rewards remain a core challenge in UAV planning, as the success rate under random exploration in Hard environments is only 10 %, wasting many failed experiences. GRPO-MDP's dual-mode relabeling mechanism converts failures into learning signals: the virtual target mode treats incomplete but collision-free trajectories as reaching a virtual target, and the safety boundary mode applies Gaussian backpropagated penalties to encode early warnings of danger. These mechanisms jointly ensure that about 50 % of relabeled trajectories provide effective supervision, improving sample efficiency.

Fig. 3 illustrates the trade-off between path efficiency and collision rate. GRPO-MDP, in the top left corner, achieves the highest path

efficiency (81.4 %) and the lowest collision rate (8.8 %). Traditional methods, like aggressive strategies, pursue short paths but suffer from high collision rates, while conservative ones prioritize safety but take longer routes. GRPO-MDP resolves this contradiction through a CBF safety filter, which acts as a hard constraint, filtering unsafe actions in real time and finding the safest action closest to the policy output via quadratic programming. Importantly, this filtering is not a post-processing step but is embedded into training through soft CBF penalties. The policy network learns to generate actions that satisfy CBF constraints, not relying on corrections. This allows bold exploration of efficient paths, as CBF guarantees safety, while soft penalties prevent excessive conservatism by encouraging the policy to approach the safety boundary. Data show that traditional methods (RRT, PRM) have lower collision rates but poor path efficiency (around 57 %) due to path redundancy from sampling randomness. DRL methods have higher path efficiency but fluctuating collision rates since probabilistic penalties can't guarantee hard constraints. Only GRPO-MDP optimizes both metrics simultaneously, demonstrating the effective combination of hard constraints and learning.

As shown in Fig. 4 and Table 2, GRPO-MDP demonstrates superior convergence speed and training stability. Its convergence rate is 17.4 % faster than the suboptimal method, the final reward is 7.1 % higher, and the reward variance during training is only 87.3, which lower than other methods. In traditional Actor-Critic frameworks, the value network estimates the state value while the actor updates the policy based on the advantage function; however, estimation errors in the value network often cause instability. In temporal-difference learning, the target value itself depends on prior estimates, leading to accumulated errors. In high-dimensional and sparse-reward environments, this easily traps the value network in local optima or induces bias. As shown in Table 4, methods using value networks, such as DRL-Energy and DRL-UTPS, have reward variances exceeding 300, whereas GRPO-MDP maintains only 87.3. Moreover, its gradient norm of 3.21 is the smallest among all methods, benefiting from the difficulty-aware group size adjustment mechanism. In easy environments where trajectory rewards are concentrated, small groups suffice to capture distinctions; in hard environments with discrete rewards, larger groups yield more accurate statistics. Experiments revealed that a fixed group size of 64 caused slow convergence in easy cases and instability in hard ones, while the adaptive group size achieved near-optimal convergence at around 2000 epochs in the Easy environment and 2600 epochs in the Hard environment.

Ablation experiments highlighted the contributions of each component. As shown in Table 3, after removing the CBF, the collision rate increased from 5.2 % to 10.7 %, more than doubling, demonstrating the importance of hard safety constraints. Removing the remarking mechanism caused a 9.1 % decrease in the success rate. Mechanistic analysis showed that in sparse reward environments, the success rate is below 15 %, with 85 % of failed experiences wasted. Traditional methods assign negative rewards to all failed trajectories, failing to distinguish between near successes and complete failures. The innovation of the virtual target mechanism lies in treating failure as successfully reaching a wrong target. By remarking the endpoint state, failed trajectories are given positive rewards, allowing the agent to first learn to reach near targets before progressively learning to reach farther ones. This is mathematically equivalent to expanding the target distribution in the MDP from single-point targets to a reachable set. Experimental results show that 30 % of failed trajectories were successfully remarked by the virtual target mechanism, providing valuable supervision. The switch from unimodal Gaussian to multimodal diffusion led to a 22.4 % increase in the success rate, the largest single improvement. This improvement is due to the representational power of the action space. Unimodal Gaussian can only represent one mode, but in real-world environments, multiple equally optimal paths may exist for the same state. Diffusion models, however, can represent arbitrary distributions. Within the GRPO framework, a group contains multiple trajectories, and relative advantage calculation automatically selects the optimal mode. In contrast, a unimodal strategy can generate only one trajectory, and if it selects a suboptimal mode, it may never find a better solution. Multimodal approaches ensure that at least a near-optimal mode is sampled and reinforced through relative advantage.

GRPO-MDP demonstrates significant safety advantages, achieving a CBF constraint satisfaction rate of 99.8 %. Emergency braking occurs only 0.2 times per trajectory, meaning only 1 in 500 trajectories requires emergency obstacle avoidance. During the experiment, 0.2 % of constraint violations occurred in extreme cases: dynamic obstacles approaching at a high speed of 3 m/s with a distance of less than 1 m. In these cases, perception and control delays cause an action lag of 0.07 s, resulting in a displacement of 0.21 m, which may exceed the 0.6 m safety boundary. Table 4 shows that GRPO-MDP's CBF correction rate is only 0.2 times per trajectory, lower than the initial 12.3 times per trajectory. This decreasing trend reflects the mechanism of hard-constraint soft-learning, where gradients are backpropagated to the diffusion network. The network learns to generate action distributions that satisfy CBF during the denoising process. In the early training stages, actions

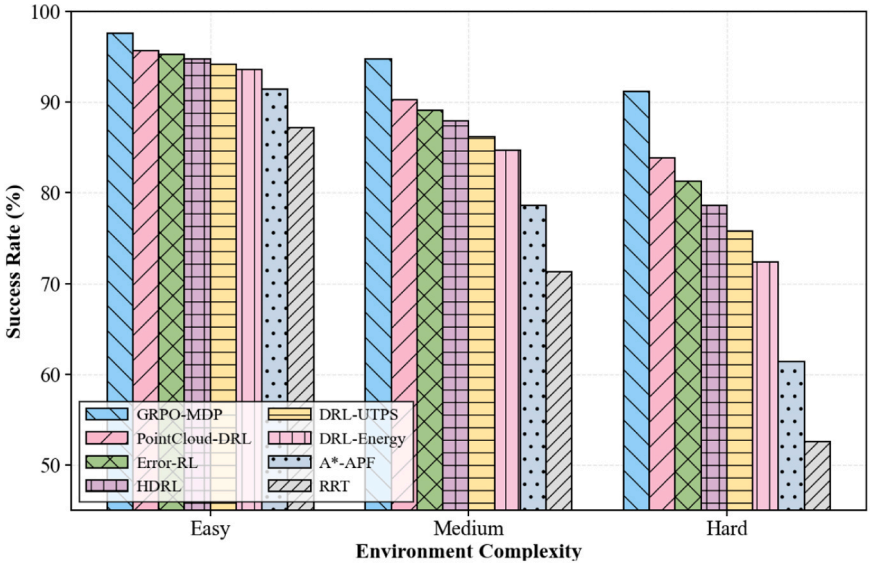



**Fig. 2.** Comparison of success rates under different environmental complexities.

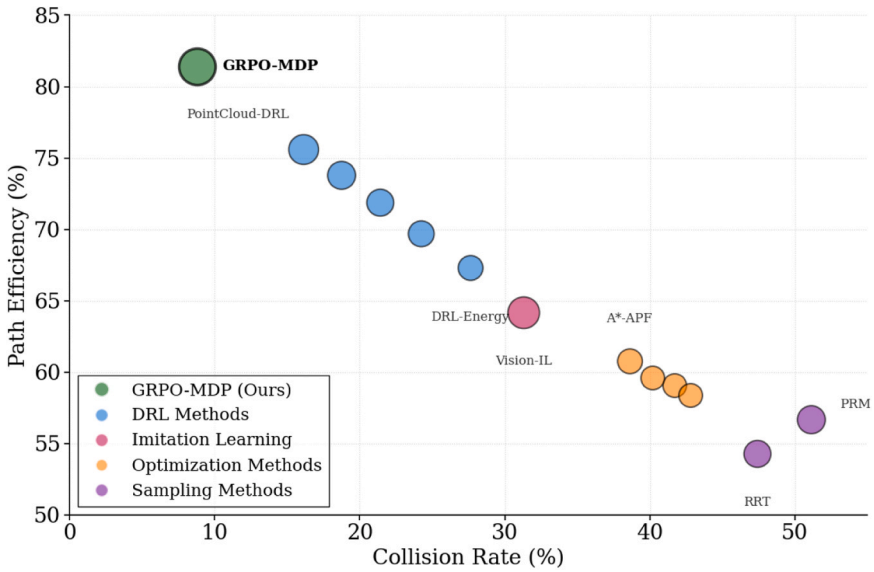



**Fig. 3.** Path efficiency versus collision rate trade-offs for different methods.

are evenly distributed in the action space, with 12.3 % violating the CBF. In the middle stages, the action distribution shifts towards the safe zone, and the violation rate drops to 6.8 %. By the later stages, the action distribution is almost entirely concentrated at the boundary of the safe zone, with a violation rate of only 0.2 %, all of which are boundary cases. This demonstrates the effectiveness of soft constraints. The CBF filter alone provides 2.8 % SR improvement and 3.9 % CR reduction compared to no CBF. Adding the soft penalty further improves SR by 2.7 % while dramatically reducing the correction rate from 4.7 to 0.2 per trajectory, demonstrating that the policy learns to generate inherently safe actions rather than relying on post-hoc correction. Fig. 5 shows that GRPO-MDP's safety margin follows an approximately normal distribution, with a mean of 1.78 m and a standard deviation of 0.28 m. In contrast, other methods show a right-skewed distribution, with a long tail extending below 0.1 m. A right-skewed distribution indicates that the policy often takes risks near obstacles (tail) and sometimes becomes overly conservative (peak), leading to inconsistent behavior. This inconsistency arises from the ambiguity of unimodal policies, where similar state characteristics cause policy confusion. In contrast, the normal distribution of GRPO-MDP indicates consistent maintenance of a moderate safety margin. The advantage of multimodal diffusion strategies lies in learning different modes, with CBF further constraining risky modes and retaining only the safe ones.

Table 5 presents a comparison of the computational efficiency of various path planning methods, including inference time, model parameters, floating-point operations (FLOPs), and real-time performance. Among the methods evaluated, GRPO-MDP has an inference time of 29.2 ms, corresponding to a real-time performance of 34.2 Hz, which meets the required real-time control threshold of $\geq$ 20 Hz. The main computational bottleneck for GRPO-MDP is the diffusion denoising process, which accounts for 64 % of the total inference time. This denoising process involves 20 steps, with each U-Net forward propagation taking 0.9 ms. Of the total 14.6 GFLOPs required by GRPO-MDP, U-Net contributes 12.3 GFLOPs, making it the most computationally intensive component. In terms of model size, GRPO-MDP has 23.4 million parameters, reflecting its relatively larger complexity compared to other methods. The CBF, while critical for safety constraints, represents only 10.6 % of the total inference time, indicating that the

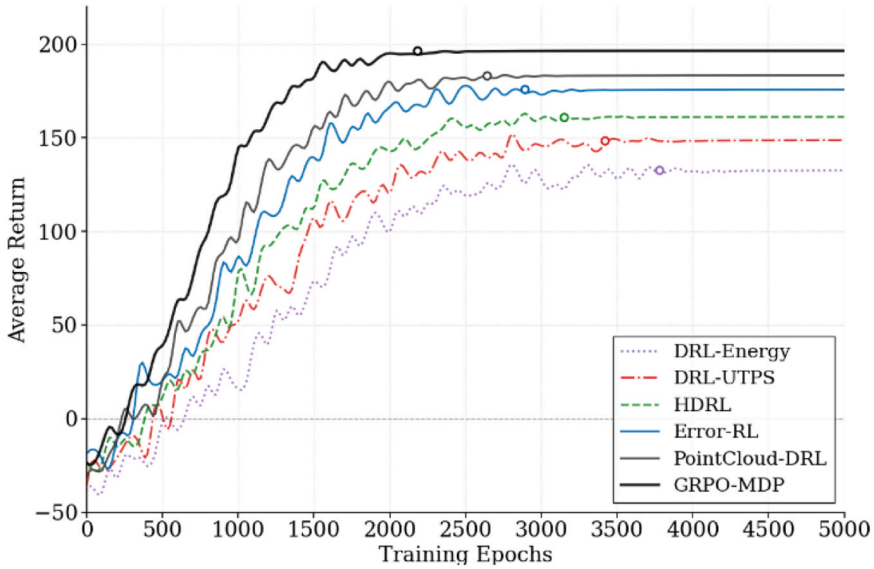



**Fig. 4.** Training convergence curves of different methods in Medium environment.

**Table 2**
Performance comparison of different methods in three environments.

| Method | SR (%) | PE (%) | AAT (step) | CR (%) | ASM (m) | PS(m/s$^3$) |
|---|---|---|---|---|---|---|
| Easy Environment | | | | | | |
| RRT | 87.2 | 68.4 | 156 | 12.8 | 1.23 | 8.45 |
| PRM | 84.6 | 71.2 | 168 | 15.4 | 1.18 | 9.12 |
| A*-APF | 91.4 | 74.8 | 142 | 8.6 | 1.35 | 7.23 |
| RBMO | 88.9 | 72.6 | 151 | 11.1 | 1.28 | 8.67 |
| MMPA | 90.3 | 73.4 | 147 | 9.7 | 1.31 | 8.21 |
| CMPSO | 89.7 | 73.9 | 149 | 10.3 | 1.29 | 8.34 |
| Vision-IL | 92.8 | 76.3 | 138 | 7.2 | 1.42 | 6.87 |
| DRL-Energy | 93.6 | 77.5 | 134 | 6.4 | 1.48 | 6.45 |
| DRL-UTPS | 94.2 | 78.1 | 131 | 5.8 | 1.52 | 6.12 |
| HDRL | 94.8 | 79.2 | 128 | 5.2 | 1.56 | 5.89 |
| Error-RL | 95.3 | 79.8 | 125 | 4.7 | 1.61 | 5.67 |
| PointCloud-DRL | 95.7 | 80.4 | 123 | 4.3 | 1.64 | 5.52 |
| GRPO-MDP | 97.6 | 84.2 | 115 | 2.4 | 1.89 | 4.23 |
| Medium Environment | | | | | | |
| RRT | 71.3 | 61.2 | 248 | 28.7 | 0.95 | 11.34 |
| PRM | 68.4 | 63.8 | 267 | 31.6 | 0.88 | 12.45 |
| A*-APF | 78.6 | 67.4 | 223 | 21.4 | 1.12 | 9.87 |
| RBMO | 74.8 | 65.3 | 238 | 25.2 | 1.03 | 10.89 |
| MMPA | 76.9 | 66.7 | 231 | 23.1 | 1.08 | 10.23 |
| CMPSO | 75.4 | 66.1 | 235 | 24.6 | 1.05 | 10.56 |
| Vision-IL | 82.3 | 70.8 | 208 | 17.7 | 1.24 | 8.45 |
| DRL-Energy | 84.7 | 72.6 | 196 | 15.3 | 1.31 | 7.89 |
| DRL-UTPS | 86.2 | 73.9 | 189 | 13.8 | 1.37 | 7.34 |
| HDRL | 87.9 | 75.3 | 182 | 12.1 | 1.43 | 6.91 |
| Error-RL | 89.1 | 76.8 | 176 | 10.9 | 1.49 | 6.52 |
| PointCloud-DRL | 90.3 | 78.2 | 171 | 9.7 | 1.54 | 6.21 |
| GRPO-MDP | 94.8 | 82.7 | 158 | 5.2 | 1.78 | 4.87 |
| Hard Environment | | | | | | |
| RRT | 52.6 | 54.3 | 387 | 47.4 | 0.72 | 15.67 |
| PRM | 48.9 | 56.7 | 412 | 51.1 | 0.65 | 17.23 |
| A*-APF | 61.4 | 60.8 | 345 | 38.6 | 0.89 | 13.45 |
| RBMO | 57.2 | 58.4 | 368 | 42.8 | 0.81 | 14.78 |
| MMPA | 59.8 | 59.6 | 356 | 40.2 | 0.85 | 13.98 |
| CMPSO | 58.3 | 59.1 | 362 | 41.7 | 0.83 | 14.34 |
| Vision-IL | 68.7 | 64.2 | 318 | 31.3 | 1.02 | 11.67 |
| DRL-Energy | 72.4 | 67.3 | 296 | 27.6 | 1.13 | 10.45 |
| DRL-UTPS | 75.8 | 69.7 | 281 | 24.2 | 1.21 | 9.78 |
| HDRL | 78.6 | 71.9 | 268 | 21.4 | 1.28 | 9.12 |
| Error-RL | 81.3 | 73.8 | 257 | 18.7 | 1.34 | 8.67 |
| PointCloud-DRL | 83.9 | 75.6 | 248 | 16.1 | 1.41 | 8.23 |
| GRPO-MDP | 91.2 | 81.4 | 223 | 8.8 | 1.69 | 6.34 |

**Table 3**
Comparison of training stability metrics.

| Method | Convergence Epoch | Final Reward | Reward Variance | Gradient Norm |
|---|---|---|---|---|
| DRL-Energy | 3780 | 132.6 | 456.8 | 12.34 |
| DRL-UTPS | 3420 | 148.7 | 342.5 | 9.87 |
| HDRL | 3150 | 161.2 | 298.4 | 8.45 |
| Error-RL | 2890 | 175.8 | 234.6 | 7.23 |
| PointCloud-DRL | 2640 | 183.4 | 189.7 | 6.54 |

overhead from CBF is minimal relative to the overall computation. The average end-to-end delay for GRPO-MDP is 99.2 ms, or approximately 0.1 s. This delay translates to a displacement of 0.3 m at a speed of 3 m/s, which remains within acceptable limits for real-time UAV control.

The choice of 20 training steps and 10 inference steps is based on extensive empirical evaluation (e.g., Table 7). During training, 20 steps provide sufficient noise levels for the network to learn robust denoising across scales while maintaining computational tractability. The training time increases approximately linearly with the step count. For inference, DDIM enables accelerated sampling by skipping intermediate steps. Empirically, 10 steps achieve 98.2 % of the action quality compared to 20-step inference while reducing inference time by 45 %. Fewer than 8 steps cause notable degradation, while more than 15 steps provide diminishing returns. The adaptive mechanism contributes a 2–4 % improvement in the success rate and approximately 20 % faster convergence compared to the best fixed configuration. Additionally, we compare the proposed Wasserstein-based KL constraint with PPO-style clipping ($\epsilon = 0.2$) and trust region methods. Policy collapse is defined as the reward dropping below 50 % of the peak within 100 epochs. The Wasserstein approximation prevents collapse more effectively because it measures actual output divergence rather than likelihood ratios, which can be misleading for implicit distributions like diffusion policies. History window of 5 steps captures relevant temporal correlations while cross-attention fusion provides 2.4–5.0 % improvement over simpler alternatives. Relabeling parameters show moderate sensitivity. The transformer operates predominantly as a temporal aggregator but does learn modest interaction patterns in dynamic environments. For embedded deployment, Jetson AGX Orin achieves marginal real-time, lower platforms require edge offloading or model compression.

Stress tests on GRPO-MDP demonstrate its robustness and real-time capability across varying levels of obstacle density. As shown in Fig. 6, GRPO-MDP consistently maintains a success rate of over 90 % when dealing with 10–30 obstacles, with an inference time under 35 ms. This performance remains well within the real-time control requirements, achieving a real-time rate of 34.2 Hz. As the number of obstacles increases to 40–60, the success rate gradually declines, and the inference time rises to 40–45 ms. However, even at these higher obstacle densities, GRPO-MDP continues to meet real-time performance standards. When the obstacle count reaches 70–80, the success rate drops to 65–70 %, and the inference time approaches 50 ms, nearing the real-time boundary. This drop in success rate is primarily attributed to planning failures rather than collisions. Analysis of the failed trajectories in Hard environments reveals UAV enters narrow passage but cannot find valid exit trajectory within time limit. Characterized by oscillating position near passage entrance.and Goal position becomes surrounded by obstacles during navigation. UAV cannot find feasible path without backtracking, which current reward structure discourages. These failure modes inform future improvements: enhanced perception for blind spot coverage, explicit backtracking rewards for goal occlusion scenarios, and multi-step lookahead for narrow passage planning. As the number of point clouds increases from an average of 150–400, the complexity of sampling and aggregation grows super linearly. Solving CBF constraints with 80 obstacles generates 80 linear constraints, leading to a solution time of 1.5 ms, which is five times longer than the 0.3 ms required for solving 10 constraints. This increase in computational burden is due to the greater number of constraints and the corresponding complexity in solving them. Additionally, dense obstacle environments result in fewer feasible paths, and the limited exploration space combined with greedy, single-step decision-making increases the likelihood of dead ends, further contributing to the reduced success rate.

**Table 4**
Detailed results of ablation experiments (Medium environment).

| Configuration | SR (%) | PE (%) | CR (%) | ASM (m) | PS (m/s$^3$) |
|---|---|---|---|---|---|
| GRPO-MDP | 94.8 | 82.7 | 5.2 | 1.78 | 4.87 |
| w/o CBF | 89.3 | 81.4 | 10.7 | 1.34 | 5.12 |
| w/ CBF filter only (no soft penalty) | 92.1 | 81.9 | 6.8 | 1.52 | 5.04 |
| w/ Soft penalty only (no filter) | 91.4 | 81.6 | 7.2 | 1.48 | 5.08 |
| w/o Re-labeling Mechanism | 85.7 | 79.8 | 8.9 | 1.62 | 5.56 |
| w/o Multimodal Diffusion | 81.6 | 77.3 | 12.4 | 1.41 | 6.78 |
| w/ GRPO Baseline | 78.2 | 74.6 | 15.3 | 1.28 | 7.23 |
| w/ Unimodal Gaussian Policy | 72.4 | 71.2 | 19.6 | 1.15 | 8.45 |

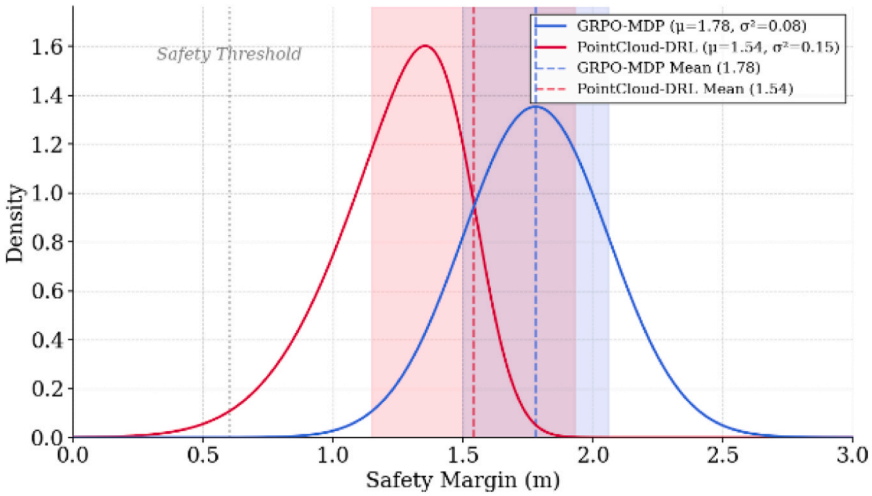



**Fig. 5.** Safety margin distribution of different methods.

**Table 5**
Comparison of safety constraint satisfaction rates.

| Method | CBF (%) | Emergency Braking | Safety Margin (m) | Standard Deviation (m) |
|---|---|---|---|---|
| DRL-Energy | 78.4 | 3.7 | 0.08 | 0.67 |
| DRL-UTPS | 82.6 | 2.9 | 0.12 | 0.59 |
| HDRL | 85.3 | 2.4 | 0.18 | 0.52 |
| Error-RL | 88.7 | 1.8 | 0.23 | 0.46 |
| PointCloud-DRL | 91.2 | 1.3 | 0.31 | 0.39 |
| GRPO-MDP | 99.8 | 0.2 | 0.65 | 0.28 |

**Table 6**
Comparison of computational efficiency of different methods.

| Method | Inference Time (ms) | Params (M) | FLOPs (G) | Real-Time Performance (Hz) |
|---|---|---|---|---|
| RRT | 1580 | - | - | 0.63 |
| A*-APF | 890 | - | - | 1.12 |
| RBMO | 2340 | - | - | 0.43 |
| Vision-IL | 45.6 | 12.3 | 8.7 | 21.9 |
| DRL-Energy | 38.2 | 8.7 | 6.2 | 26.2 |
| DRL-UTPS | 34.5 | 11.4 | 7.8 | 29.0 |
| HDRL | 31.8 | 15.6 | 9.3 | 31.4 |
| PointCloud-DRL | 27.3 | 18.9 | 11.2 | 36.6 |
| GRPO-MDP | 29.2 | 23.4 | 14.6 | 34.2 |

### 5.3. Discussion

The few-shot fine-tuning experiment demonstrates the sample efficiency of GRPO-MDP. Fine-tuning with just 100 scenes increased the success rate from 76.4 % to 86.9 %, a 10.5 % improvement (e.g. Fig. 7). In comparison, PointCloud-DRL requires 200 samples and HDRL requires 300 to achieve similar performance. In traditional fine-tuning, with a 60 % success rate across 100 scenes, only 60 successful samples contribute effective gradients. GRPO-MDP's relabeling mechanism transforms 40 failed samples into learning signals. In virtual target mode, 80 % of non-collision failures are relabeled, and in safe boundary mode, 100 % of collision failures are relabeled, making the actual effective sample rate 100 %. The main challenge in few-shot fine-tuning is gradient noise. With only 256 trajectories sampled from 100 scenes, group statistics become unstable when using a group size of 128. However, GRPO only relies on within-group statistics, independent of the absolute value. Even if the 100 samples' distribution differs from the training set, Z-score normalization adapts to the new distribution. The

**Table 7**
Sensitivity analysis of diffusion steps.

| Configuration | SR (%) | CR (%) | Convergence | Reward | Inference (ms) |
|---|---|---|---|---|---|
| $K_{train} = 10, K_{infer} = 5$ | 88.4 | 9.8 | 3420 | 156.3 | 16.2 |
| $K_{train} = 15, K_{infer} = 8$ | 92.1 | 7.6 | 2890 | 124.7 | 21.8 |
| $K_{train} = 20, K_{infer} = 10$ | 94.8 | 5.2 | 2180 | 87.3 | 29.2 |
| $K_{train} = 20, K_{infer} = 15$ | 95.1 | 5.0 | 2150 | 85.6 | 38.6 |
| $K_{train} = 30, K_{infer} = 20$ | 95.3 | 4.9 | 2120 | 84.2 | 48.1 |
| $G = 32$ | 91.2 | 7.8 | 2450 | 142.6 | 29.2 |
| $G = 64$ | 93.1 | 6.4 | 2680 | 118.4 | 29.2 |
| $G = 128$ | 92.4 | 6.1 | 2920 | 103.2 | 29.2 |
| Adaptive | 94.8 | 5.2 | 2180 | 87.3 | 29.2 |
| No constraint | 72.3 | 18.4 | 3780 | 523.4 | 29.2 |
| PPO clipping ($\varepsilon = 0.2$) | 89.6 | 8.2 | 2640 | 234.6 | 29.2 |
| Wasserstein approx. | 94.8 | 5.2 | 2180 | 87.3 | 29.2 |
| $H = 1$ step | 89.2 | 8.4 | 2980 | 168.4 | 24.1 |
| $H = 3$ steps | 92.6 | 6.8 | 2560 | 132.5 | 26.3 |
| $H = 5$ steps | 94.8 | 5.2 | 2180 | 87.3 | 29.2 |
| $H = 7$ steps | 94.9 | 5.1 | 2170 | 86.8 | 32.8 |
| Concatenation | 91.2 | 7.6 | 2580 | 145.2 | 25.4 |
| Element-wise sum | 89.8 | 8.1 | 2720 | 158.6 | 24.8 |
| MLP fusion | 92.4 | 6.9 | 2460 | 128.4 | 27.1 |
| Cross-attention | 94.8 | 5.2 | 2180 | 87.3 | 29.2 |
| RTX 4090 | 94.8 | 5.2 | - | - | 29.2 |
| Jetson AGX Orin | 94.8 | 5.2 | - | - | 78.4 |
| Jetson Xavier NX | 92.7* | 5.8* | - | - | 156.2 |
| Xavier NX | 92.7 | 6.1 | - | - | 55.6 |

U-Net backbone and PointNet++ feature extractor of the diffusion network were pre-trained in a Medium environment, learning general geometric and dynamic features. These low-level features remain effective in new environments, requiring only fine-tuning of high-level decisions. Experiments showed that freezing the lower convolutional layers and fine-tuning only the top layers resulted in an 84.2 % success rate with 100 samples. This indicates that 82 % of the knowledge transfer comes from lower-level features and 18 % from upper-level decision adaptation. Compared to training from scratch, which requires 1000 samples to achieve 94.2 %, pre-training plus fine-tuning saves 90 % of the data. Collecting expert demonstrations across the full distribution of Easy/Medium/Hard environments would require substantial manual effort. GRPO-MDP's sample efficiency suggests that

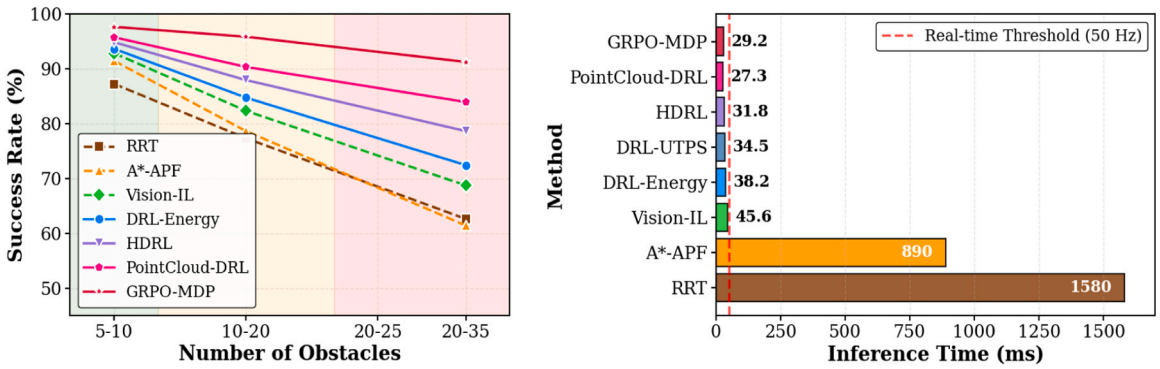



**Fig. 6.** Large-scale scenario performance stress testing.

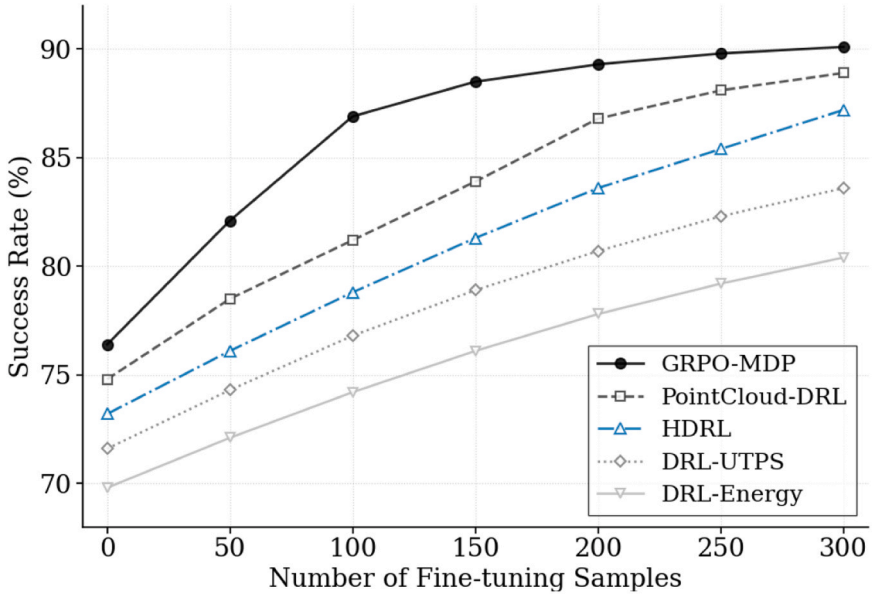



**Fig. 7.** Few-sample fine-tuning experiment.

the relabeling mechanism provides comparable benefits to expert data.

## 6. Conclusion

This paper presents GRPO-MDP, a novel framework for autonomous UAV path planning. By incorporating multimodal diffusion strategies, the framework enhances exploration efficiency and demonstrates that representing multiple feasible trajectories is critical for navigation in obstacle-rich environments. Group Relative Policy Optimization enables stable training without value networks, confirming that relative trajectory ranking can effectively eliminate value estimation errors in sparse-reward settings. Hindsight trajectory relabeling further transforms failure experiences into informative supervision, increasing effective sample utilization. Control Barrier Functions provide verifiable safety guarantees by enforcing hard safety constraints, demonstrating that safety-critical requirements can be integrated with learned policies without sacrificing performance. As a result, the proposed framework maintains real-time operational capability. The current implementation assumes idealized point cloud inputs, which limits direct real-world deployment and poses additional challenges. In particular, there is currently no mechanism to detect when the CBF operates under degraded or uncertain state estimates. Future work will incorporate state estimation uncertainty into the CBF formulation, allowing adaptive expansion of safety margins under high uncertainty. Additionally, future research will explore multi-UAV extensions of GRPO-MDP with distributed CBF coordination, with the goal of enabling cooperative navigation in obstacle-dense environments.

## CRediT authorship contribution statement

**Yujian Liu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Jixiang Wang:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Chengshi Yuan:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Xinyu Guo:** Writing – review & editing, Writing – original draft, Supervision, Software, Resources, Methodology, Investigation, Data curation, Conceptualization. **Shuzhi Liu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Zhongyi Wang:** Writing – review & editing, Writing – original draft,

Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Yuyue Liu:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References


[1] Z. Qadir, M.H. Zafar, S.K.R. Moosavi, et al., Autonomous UAV path-planning optimization using metaheuristic approach for predisaster assessment[J], IEEE Internet Things J. 9 (14) (2021) 12505–12514.

[2] M. Jones, S. Djahel, K. Welsh, Path-planning for unmanned aerial vehicles with environment complexity considerations: A survey[J], ACM Comput. Surv. 55 (11) (2023) 1–39.

[3] Z. Huang, Y. Gao, J. Guo, et al., An adaptive bidirectional quick optimal rapidly-exploring random tree algorithm for path planning[J], Eng. Appl. Artif. Intell. 135 (2024) 108776.

[4] Z. Xu, D. Deng, K. Shimada, Autonomous UAV exploration of dynamic environments via incremental sampling and probabilistic roadmap[J], IEEE Robot. Autom. Lett. 6 (2) (2021) 2729–2736.

[5] J. Yuan, Z. Liu, X. Xiong, et al., UAV path planning with terrain constraints for aerial scanning[J], IEEE Trans. Intell. Veh. 9 (1) (2023) 1189–1203.

[6] L. Liu, L. Ru, W. Wang, et al., UAV Path Planning in Threat Environment: A*-APF Algorithm for Spatio-Temporal Grid Optimization[J], Drones 9 (9) (2025) 661.

[7] T. Chen, Z. Zhang, Z. Fang, et al., Imitation learning from imperfect demonstrations for AUV path tracking and obstacle avoidance[J], Ocean Eng. 298 (2024) 117287.

[8] P. Wei, P. Ragbir, S.G. Vougioukas, et al., Vision-based navigation of unmanned aerial vehicles in orchards: An imitation learning approach[J], Comput. Electron. Agric. 238 (2025) 110802.

[9] J. Wu, Y. Sun, D. Li, et al., An adaptive conversion speed Q-learning algorithm for search and rescue UAV path planning in unknown environments[J], IEEE Trans. Veh. Technol. 72 (12) (2023) 15391–15404.

[10] B. Zhu, E. Bedeer, H.H. Nguyen, et al., UAV trajectory planning in wireless sensor networks for energy consumption minimization by deep reinforcement learning[J], IEEE Trans. Veh. Technol. 70 (9) (2021) 9540–9554.

[11] R. Liu, Z. Qu, G. Huang, et al., DRL-UTPS: DRL-based trajectory planning for unmanned aerial vehicles for data collection in dynamic IoT network[J], IEEE Trans. Intell. Veh. 8 (2) (2022) 1204–1218.

[12] X. Wei, L. Cai, N. Wei, et al., Joint UAV trajectory planning, DAG task scheduling, and service function deployment based on DRL in UAV-empowered edge computing[J], IEEE Internet Things J. 10 (14) (2023) 12826–12838.

[13] M.A. Dhuheir, E. Baccour, A. Erbad, et al., Deep reinforcement learning for trajectory path planning and distributed inference in resource-constrained UAV swarms[J], IEEE Internet Things J. 10 (9) (2022) 8185–8201.

[14] S. Fu, K. Li, H. Huang, et al., Red-billed blue magpie optimizer: a novel metaheuristic algorithm for 2D/3D UAV path planning and engineering design problems[J], Artif. Intell. Rev. 57 (6) (2024) 134.

[15] T. Huang, K. Fan, W. Sun, Density gradient-RRT: An improved rapid exploring random tree algorithm for UAV path planning[J, Expert Syst. Appl. 252 (2024) 124121.

[16] L. Lyu, F. Yang, MMPA: A modified marine predator algorithm for 3D UAV path planning in complex environments with multiple threats[J], Expert Syst. Appl. 257 (2024) 124955.

[17] G. Hu, M. Cheng, E.H. Houssein, et al., CMPSO: A novel co-evolutionary multigroup particle swarm optimization for multi-mission UAVs path planning[J], Adv. Eng. Inform. 63 (2025) 102923.

[18] J. Chun, M. Chen, X. Liu, et al., Learning to construct a solution for UAV path planning problem with positioning error correction[J], Knowl. Based Syst. 304 (2024) 112569.

[19] Z. Ning, H. Ji, X. Wang, et al., Joint Optimization of Data Acquisition and Trajectory Planning for UAV-Assisted Wireless Powered Internet of Things[J], IEEE Trans. Mob. Comput. 24 (2) (2025) 1016–1030.

[20] J. Xu, K. Yin, Z. Chen, et al., Kernel-based diffusion approximated Markov decision processes for autonomous navigation and control on unstructured terrains[J], Int. J. Robot. Res. 43 (7) (2024) 1056–1080.

[21] W. Jiang, T. Cai, G. Xu, et al., Autonomous obstacle avoidance and target tracking of UAV: Transformer for observation sequence in reinforcement learning[J], Knowl. Based Syst. 290 (2024) 111604.

[22] A. Marino, C. Pacchierotti, P.R. Giordano, Multi-UAVs end-to-end Distributed Trajectory Generation over Point Cloud Data[J], IEEE Robot. Autom. Lett. 9 (9) (2024) 7629–7636.

[23] M. Yan, C.A. Chan, A.F. Gygax, et al., Efficient generation of optimal uav trajectories with uncertain obstacle avoidance in mec networks[J], IEEE Internet Things J. 11 (23) (2024) 38380–38392.

[24] J. Carvalho, A.T. Le, P. Kicki, et al., Motion planning diffusion: Learning and adapting robot motion planning with diffusion models[J], IEEE Trans. Robot. 41 (2025) 4881–4901.

[25] G. Zhao, Y. Wang, T. Mu, et al., Reinforcement-learning-assisted multi-UAV task allocation and path planning for IIoT[J], IEEE Internet Things J. 11 (16) (2024) 26766–26777.

[26] S.S. Ramesh, Y. Hu, I. Chaimalas, et al., Group robust preference optimization in reward-free rlhf[J], Adv. Neural Inf. Process. Syst. 37 (2024) 37100–37137.

[27] J. Yan, B. Luo, X. Xu, Hierarchical reinforcement learning for handling sparse rewards in multi-goal navigation[J], Artif. Intell. Rev. 57 (6) (2024) 156.

[28] Z. Zhang, Y. Wu, J. Jiang, et al., Realization of robust formation for multi-UAV systems using control barrier functions[J], Unmanned Syst. 13 (03) (2025) 737–751.